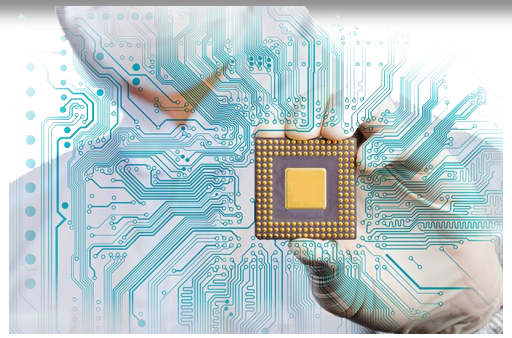
MÁQUINAS DE ESTADO EN VHDL

PRINCIPIO DE DISEÑO VLSI
DISEÑO DIGITAL CON TÉCNICAS DE ALTO NIVEL
ÁREA DISPOSITIVOS
DEPTO. DE ELECTRÓNICA Y COMPUTACIÓN
FI.UNMdP

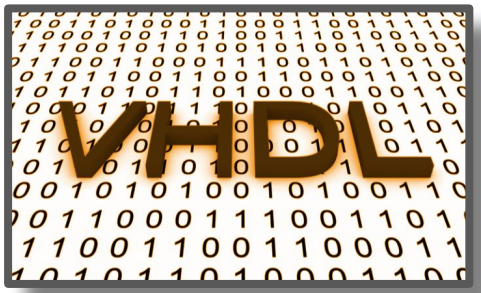


UNIVERSIDAD NACIONAL DE MAR DEL PLATA

FACULTAD DE INGENIERIA

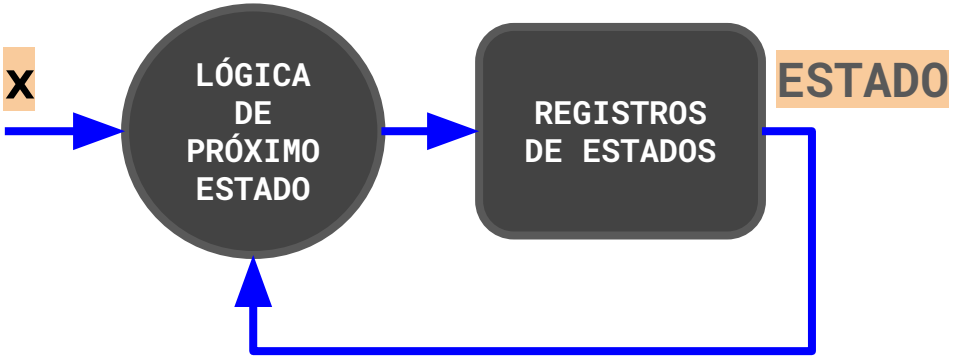
MÁQUINAS DE ESTADO

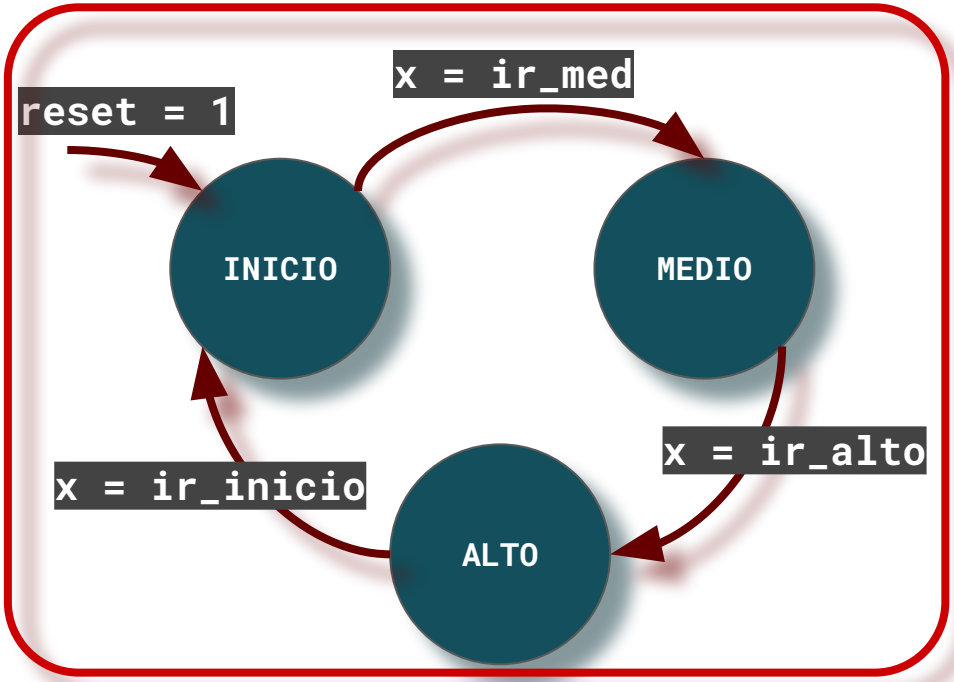
- MÁQUINAS DE ESTADO EN 1 PROCESO
- MÁQUINAS DE ESTADO EN 2 PROCESOS
- FSM MEDVEDEV
- FSM MOORE
- FSM MEALY
- FSM COMPARACIÓN



MÁQUINAS DE ESTADO EN 1 PROCESO

```
type e is (inicio, medio, alto);
signal estado : e;
```

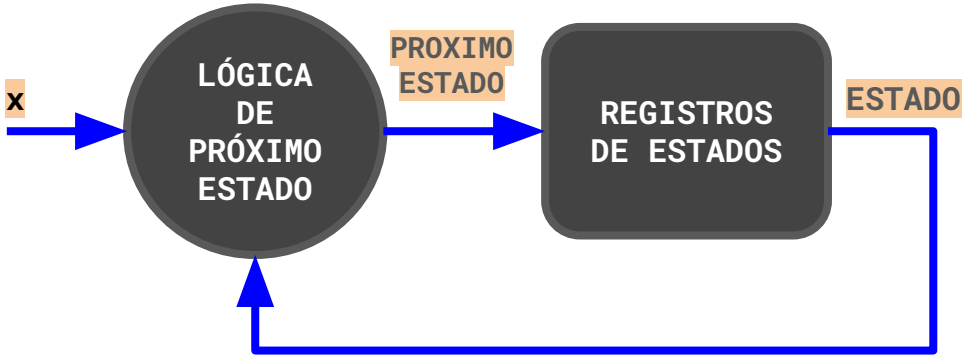


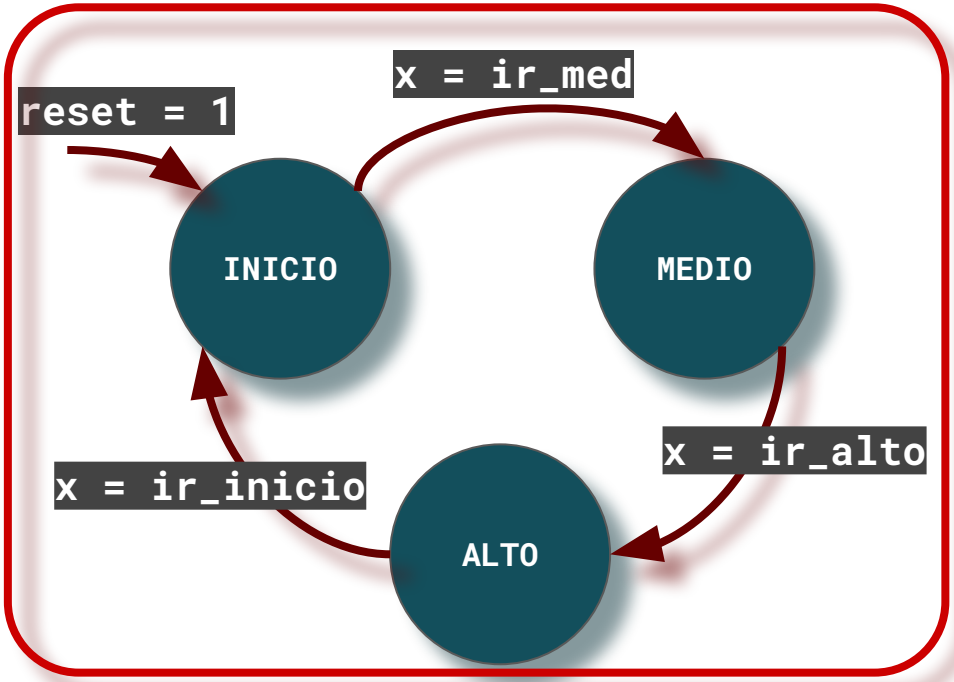



```
process (clk, reset)
begin
  if reset = '1' then
    estado <= inicio;
  elsif clk'event and clk='1' then
    case estado is
      when inicio =>
        if x = ir_med then
          estado <= medio;
        end if ;
      when medio =>
        if x = ir_alto then
          estado <= alto;
        end if;
      when alto =>
        if x = ir_inicio then
          estado <= inicio;
        end if;
      when others =>
        estado <= inicio;
    end case;
  end if;
end process;
```

MÁQUINAS DE ESTADO EN 2 PROCESOS

```
type e is (inicio, medio, alto);
signal estado, proximo_estado : e;
```

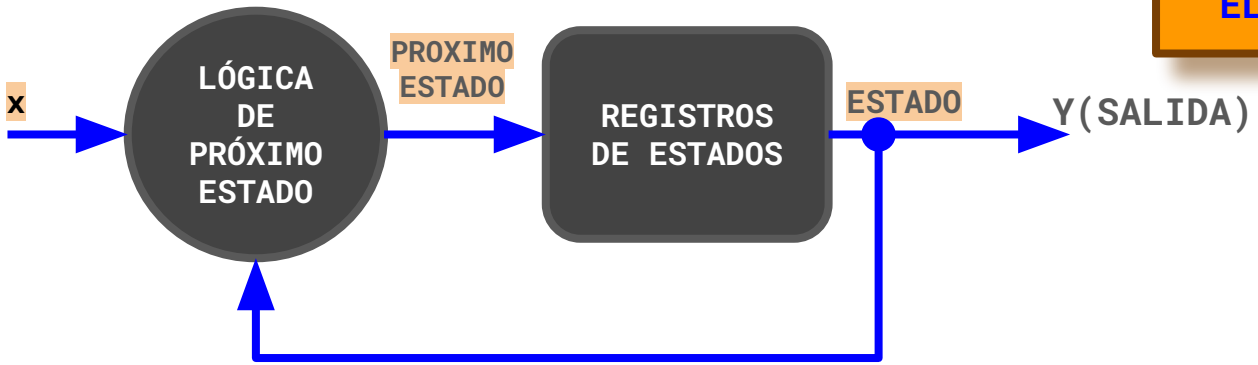





```
process (clk, reset)
begin
  if reset= '1' then
    estado <= inicio;
  elsif clk'event and clk='1' then
    estado <= proximo_estado;
  end if;
end process;

process (estado,x)
begin
  case estado is
    when inicio =>
      if x = ir_med then
      proximo_estado <= medio;
      end if;
    when medio =>
      if x = ir_alto then
      proximo_estado <= alto;
      end if;
    when alto =>
      if x = ir_inicio then
      proximo_estado <= inicio;
      end if;
    when others =>
      proximo_estado <= inicio;
  end case;
end process;
```

MÁQUINA DE ESTADOS TIPO MEDVEDEV



EL VECTOR DE SALIDA ES EL VECTOR DE ESTADOS

DOS PROCESOS

```
architecture rtl of medvedev is
...
begin
  process (clk, reset)
  begin
    -- inferencia de los registros de estado
  end process;
  process (x, state)
  begin
    -- lógica de próximo estado
  end process cmb;
  y <= s;
end rtl;
```

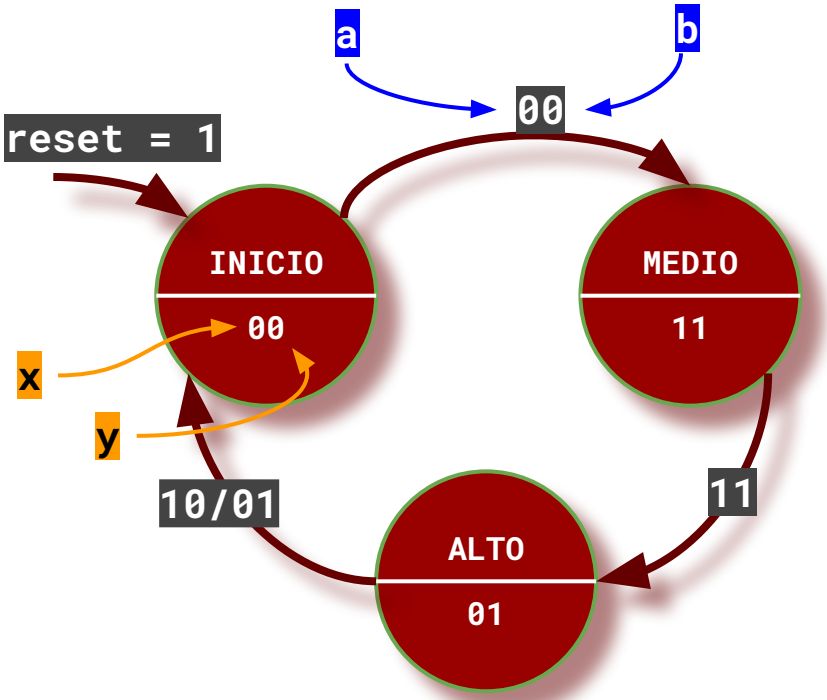
UN PROCESO

```
architecture rtl of medvedev is
...
begin
    process (clk, reset)
    begin
        -- inferencia de registros de
        estado con bloques lógicos.
    end process reg;
    y <= s;
end rtl;
```

MÁQUINA DE ESTADOS TIPO MEDVEDEV

```
subtype tipo_estado is std_ulogic_vector(1 downto 0);
constant inicio : tipo_estado =“00”;
constant medio : tipo_estado =“11”;
constant alto : tipo_estado =“01”;
```

SI NO SE ASIGNAN ESTADOS, LA HERRAMIENTA LO HACE AUTOMÁTICAMENTE



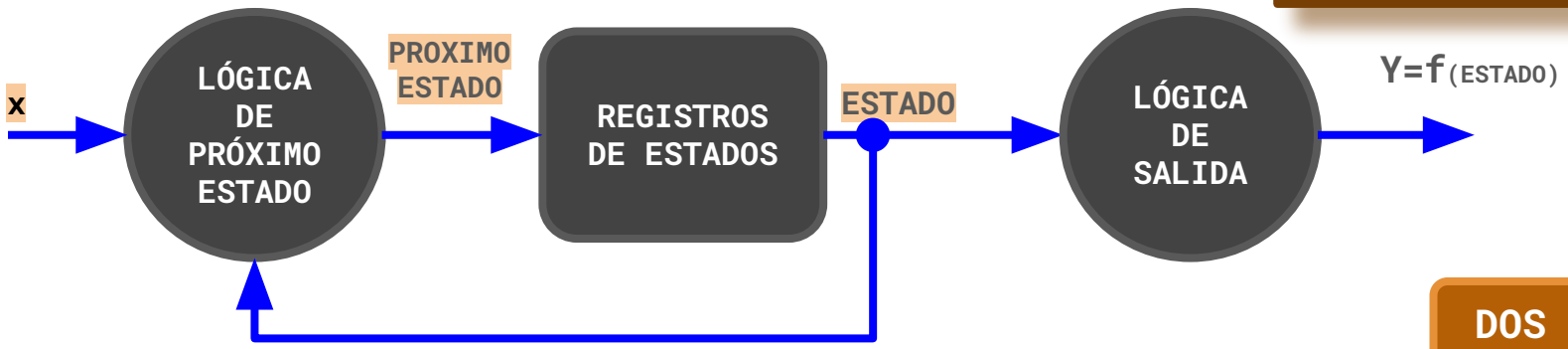
EJEMPLO

```
architecture rtl of medvedev_test is
  signal estado, proximo_estado : tipo_estado;
begin
  process (clk, reset)
  begin
    if reset=`1` then
      estado <= inicio;
    elsif clk`event and clk=`1` then
      estado <= proximo_estado;
    end if ;
  end process;

  process (a, b, estado) begin
    case estado is
      when inicio => if (a aor b)=`0` then
        proximo_estado <= medio;
      end if ;
      when medio => if (a and b)=`1` then
        proximo_estado <= alto;
      end if ;
      when alto => if (a xor b)=`1` then
        proximo_estado <= inicio;
      end if ;
      when others =>proximo_estado <= inicio;
    end case ;
  end process cmb ;
  (x, y) <= estado;
end rtl;
```

MÁQUINA DE ESTADOS TIPO MOORE

EL VECTOR DE SALIDA ES FUNCIÓN DEL VECTOR DE ESTADOS



TRES PROCESOS

```
architecture rtl of moore is
...
begin
  reg: -- proceso sensible reloj
  cmb: -- proceso combinacional
  process (estado)
  begin
    -- lógica de salida
  end process;
end rtl;
```

DOS PROCESOS

```
architecture rtl of moore is
...
begin
  process (clk, reset)
  begin
    -- inferencia de registros de
    estado con próximo estado
  end process reg ;
  process (estado)
  begin
    -- lógica de salida
  end process;
end rtl;
```

MÁQUINA DE ESTADOS TIPO MOORE

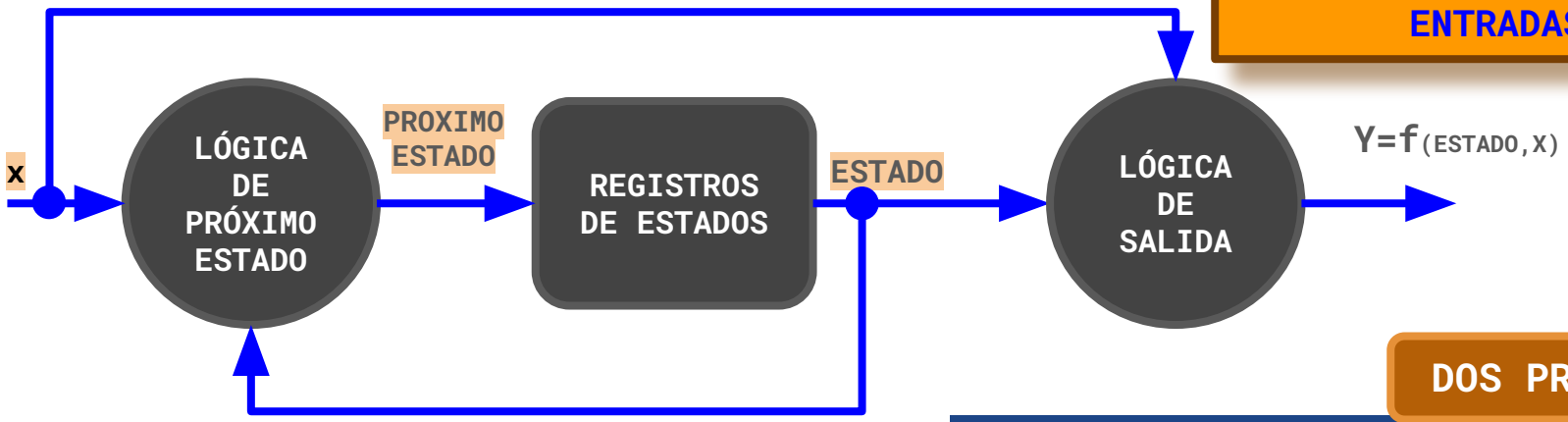
```
architecture rtl of moore_test is
    signal estado : tipo_estado;
    signal proximo_estado : tipo_estado;
begin
    process (clk, reset) begin
        if reset=`1` then
             estado <= inicio;
        elsif clk`event and clk=`1` then
             estado <= proximo_estado;
        end if;
    end process;
```

```
subtype tipo_estado is std_ulogic_vector(1 downto 0);
constant inicio : tipo_estado =“00”;
constant medio : tipo_estado =“01”;
constant alto : tipo_estado =“10”;
```

```
    process (a, b, estado)
    begin
        case estado is
            when inicio =>
                if (a or b)=`0` then
                    proximo_estado <= medio;
                end if;
            when medio =>
                if (a and b)=`1` then
                    proximo_estado <= alto;
                end if;
            when alto =>
                if (a xor b)=`1` then
                    proximo_estado <= inicio;
                end if;
            when others =>
                proximo_estado <= inicio;
        end case;
    end process;
    y <= ‘1’ when estado = medio else ‘0’;
    z <= ‘1’ when estado = medio or estado = alto
    else ‘0’;
end rtl ;
```

MÁQUINA DE ESTADOS TIPO MEALY

EL VECTOR DE SALIDA ES FUNCIÓN DEL VECTOR DE ESTADOS Y DE LAS ENTRADAS



TRES PROCESOS

```
architecture rtl of mealy is
...
begin
  reg: -- proceso sensible a reloj
  cmb: -- proceso combinacional

  process(estado,x)
  begin
    -- lógica de salida
  end process;
end rtl ;
```

DOS PROCESOS

```
architecture rtl of mealy is
...
begin

  process(clk,reset)
  begin
    -- inferencia de registros de
    estado lógica de próximo estado
  end process med;

  process(state,x)
  begin
    -- output logic
  end process output;
end rtl ;
```

SÍNTESIS DE FSM
COMPARACIÓN

- **MEDVEDEV:**
  - ARQUITECTURA DEMASIADO RÍGIDA
- **MOORE:**
  - OPERACIÓN MÁS SEGURA
- **MEALY:**
  - LA MÁS FLEXIBLE
  - PULSOS ESPURIOS
  - PUEDE TENER MENOS ESTADOS. (EL COSTO FINAL PUEDE SER MENOR)